

Stack and Queue Questions

Question 1: Balanced Parentheses Using Stack

Problem Statement: Given a string of parentheses () and [], determine if the string is balanced. A string is balanced if every opening bracket has a corresponding closing bracket in the correct order.

Example Input: "[(() []])"

Expected Output: True

Code Solution:

```
def is_balanced(expression):
    stack = []
    matching_brackets = {'(': ')', '[': ']', '{': '}'

    for char in expression:
        if char in "([{" : # If it's an opening bracket, push to stack
            stack.append(char)
        elif char in ")]}" : # If it's a closing bracket
            if not stack or stack[-1] != matching_brackets[char]:
                return False
            stack.pop() # Pop the matching opening bracket

    return len(stack) == 0 # If stack is empty, it's balanced
```

Dry Run:

- Input: "[(() []])" - Stack initially: []

Character	Action	Stack Status
(	Push	[(]
[	Push	[([]
(	Push	[([(]
)	Pop	[([]
[	Push	[([[]
]	Pop	[([]
)	Pop	[(]
]	Pop	[]

Explanation:

1. We start with an empty stack.
2. We process the first character (and push it onto the stack.
3. The next character [is also pushed onto the stack.
4. The third character (is pushed onto the stack.
5. The fourth character) is a closing bracket. It matches the top of the stack (, so we pop the stack.
6. The fifth character [is pushed onto the stack.
7. The sixth character] is a closing bracket. It matches the top of the stack [, so we pop the stack.
8. The seventh character) is a closing bracket. It matches the top of the stack (, so we pop the stack.
9. The eighth character] is a closing bracket. It matches the top of the stack [, so we pop the stack.
10. The stack is now empty, indicating that the string is balanced.

Question 2: Reverse a Queue

Problem Statement: Given a queue, reverse the order of its elements using a stack.

Example Input: Queue: [1, 2, 3, 4, 5]

Expected Output: Queue: [5, 4, 3, 2, 1]

Code Solution:

```
from collections import deque

def reverse_queue(queue):
    stack = []

    # Transfer elements from queue to stack
    while queue:
        stack.append(queue.popleft())

    # Transfer elements back to the queue from stack
    while stack:
        queue.append(stack.pop())

    return queue
```

Dry Run:

- **Input:** Queue [1, 2, 3, 4, 5] - **Initially:** queue = deque([1, 2, 3, 4, 5]), stack = []

Step	Queue Status	Stack Status
Move 1 to stack	[2, 3, 4, 5]	[1]
Move 2 to stack	[3, 4, 5]	[1, 2]
Move 3 to stack	[4, 5]	[1, 2, 3]
Move 4 to stack	[5]	[1, 2, 3, 4]
Move 5 to stack	[]	[1, 2, 3, 4, 5]
Move 5 to queue	[5]	[1, 2, 3, 4]
Move 4 to queue	[5, 4]	[1, 2, 3]
Move 3 to queue	[5, 4, 3]	[1, 2]
Move 2 to queue	[5, 4, 3, 2]	[1]
Move 1 to queue	[5, 4, 3, 2, 1]	[]

Explanation:

1. We start with the queue [1, 2, 3, 4, 5] and an empty stack.
2. We move each element from the queue to the stack, one by one. After this process, the queue is empty and the stack contains [1, 2, 3, 4, 5].
3. Next, we move each element back from the stack to the queue. This reverses the order of the elements in the queue, resulting in [5, 4, 3, 2, 1].
4. The final queue is [5, 4, 3, 2, 1], which is the reversed order of the original queue.

Question 3: Next Greater Element Using Stack

Problem Statement: Given an array, for each element find the next greater element. The next greater element is the first element to the right that is greater than the current element. If no such element exists, return -1 for that position.

Example Input: [4, 5, 2, 10, 8]

Expected Output: [5, 10, 10, -1, -1]

Code Solution:

```
def next_greater_element(arr):
    stack = []
    result = [-1] * len(arr)

    for i in range(len(arr)):
        while stack and arr[stack[-1]] < arr[i]:
            result[stack.pop()] = arr[i]
        stack.append(i)

    return result
```

Dry Run:

- **Input:** [4, 5, 2, 10, 8] - **Stack initially:** [], **Result initially:** [-1, -1, -1, -1, -1]

Step	Element	Stack Status	Result Status
1	4	[0]	[-1, -1, -1, -1, -1]
2	5	[]	[5, -1, -1, -1, -1]
3	2	[2]	[5, -1, -1, -1, -1]
4	10	[]	[5, 10, 10, -1, -1]
5	8	[3, 4]	[5, 10, 10, -1, -1]

Explanation:

1. We initialize a stack and the result array with -1.
2. We process the first element 4. Since the stack is empty, we push the index 0 onto the stack.
3. The next element 5 is greater than 4. We pop the stack and set `result[0] = 5`. We then push the index 1 onto the stack.
4. The third element 2 is less than 5. We push its index 2 onto the stack.
5. The fourth element 10 is greater than 2 and 5. We pop the stack twice, setting `result[2] = 10` and `result[1] = 10`. We push the index 3 onto the stack.
6. The last element 8 is less than 10. We push its index 4 onto the stack.
7. The stack is now [3, 4], and there are no more elements to process, so the result remains as [-1, -1, -1, -1, -1]. The final result is [5, 10, 10, -1, -1].